



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algorithms and Data Structures I.

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

March 15, 2026

Content in this Practice I

1 Divide and Conquer algorithms

- Partition
- Combine
- Conquer

2 Quicksort in Detail

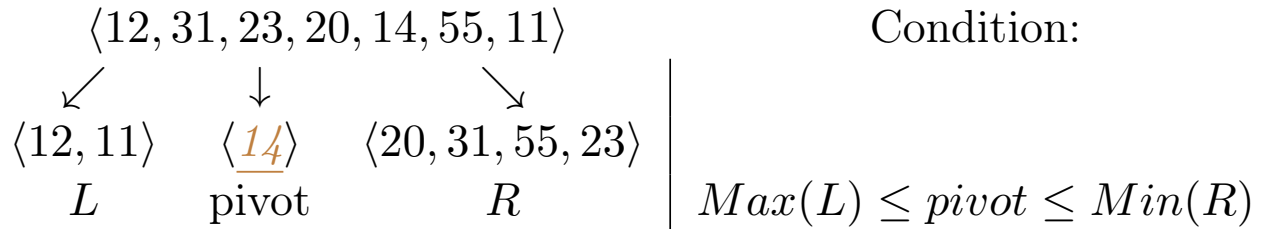
- Partitioning Process
- Step by step
- Quicksort - Structogram
- Quicksort - Summary
- Array detailed example

Divide and Conquer algorithms

- QuickSort: *a sorting algorithm based on the "Divide and Conquer" algorithm that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.*
- QuickSort:
 - Choose a pivot element, place the pivot at its correct position in the sorted array, and put all smaller elements to the left of the pivot, and all greater elements to the right of the pivot.
 - Partition is done recursively on each side of the pivot after the pivot is placed in its correct position and this finally sorts the array.

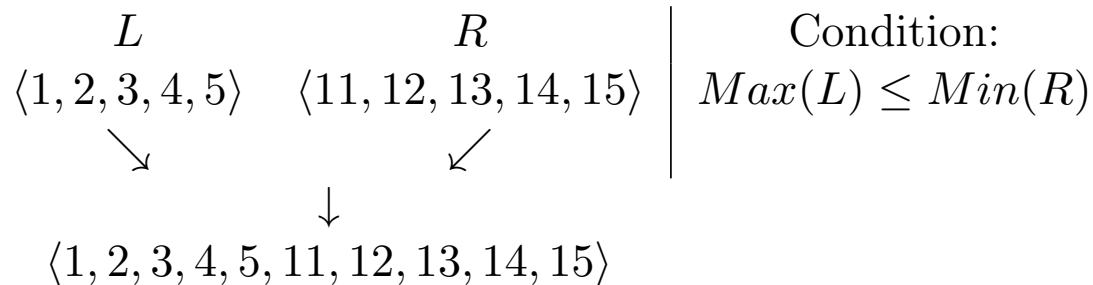
Overview: Partition

- I divide the given $A[p..r]$ array into 2 subarrays using a pivot element $A[q]$, into the *left* $L := A[p..q - 1]$, and the *right* $R := A[q + 1..r]$ subarrays such that every element of L is less than or equal to $A[q]$, and the element $A[q]$ is less than or equal to every element of R .



Overview: Combine

- If the *left* and the *right* subarrays are sorted, and every element of the *left* subarray is smaller than any element of the *right* subarray, then they can be merged without additional steps.



- How do we obtain sorted L and R arrays?

Overview: Conquer

- The subarrays $A[p..q - 1]$ and $A[q + 1..r]$ are sorted using *recursive* calls of quicksort.
- This is repeated until we obtain subarrays of size 1, or possibly 0, which are sorted.
- Since the elements of the subarrays are placed into their correct positions by the end of the recursion, the merging step does not need to be performed; we "simply" obtain the solution in the array.

Quicksort in Detail

Quicksort - Partitioning Process

Array:	$\langle 12, 31, 23, 20, 14, 55, 11 \rangle$
Choosing the <i>PIVOT</i> item:	$\langle 12, 31, 23, 20, \underline{14}, 55, 11 \rangle$
Swap with the last element:	$\langle 12, 31, 23, 20, 11, 55, \underline{14} \rangle$
Look for $A[i] > \textit{PIVOT}$:	$\langle 12, \underline{31}, 23, 20, 11, 55, \underline{14} \rangle$
Set $j := i + 1$:	$\langle 12, \underline{31}, \underline{23}, 20, 11, 55, \underline{14} \rangle$
Look for $A[j] < \textit{PIVOT}$:	$\langle 12, \underline{31}, 23, 20, \underline{11}, 55, \underline{14} \rangle$
SWAP $A[i]$ and $A[j]$:	$\langle 12, \underline{11}, 23, 20, \underline{31}, 55, \underline{14} \rangle$
Set $i := i + 1$:	$\langle 12, 11, \underline{23}, 20, \underline{31}, 55, \underline{14} \rangle$
Continue $A[j] < \textit{PIVOT}$:	$\langle 12, 11, \underline{23}, 20, \underline{31}, 55, \underline{14} \rangle$
j reaches the <i>PIVOT</i> :	$\langle 12, 11, \underline{23}, 20, 31, 55, \underline{14} \rangle$
SWAP $A[i]$ with the <i>PIVOT</i> :	$\langle 12, 11, \underline{14}, 20, 31, 55, \underline{23} \rangle$
$\langle A \text{ (small)} \rangle \textit{PIVOT} \langle B \text{ (big)} \rangle$:	$\langle 12, 11 \rangle \underline{14} \langle 20, 31, 55, 23 \rangle$

Quicksort - Initialize PIVOT

Array:	$\langle 12, 31, 23, 20, 14, 55, 11 \rangle$
Choosing the <i>PIVOT</i> item:	$\langle 12, 31, 23, 20, 14, 55, 11 \rangle$
Swap with the last element:	$\langle 12, 31, 23, 20, 11, 55, 14 \rangle$

$\text{partition}(A : \mathcal{T}[] ; p, r : \mathbb{N}) : \mathbb{N}$

$i := \text{random}(p, r)$ // Select a random pivot

$\text{swap}(A[i], A[r])$ // $A[r]$ is the pivot

$i := p$

Quicksort - Looking for larger elements

Look for $A[i] > PIVOT$:	$\langle 12, 31, 23, 20, 11, 55, 14 \rangle$
$i := p$	
$i < r \wedge A[i] \leq A[r]$	
	$i := i + 1$

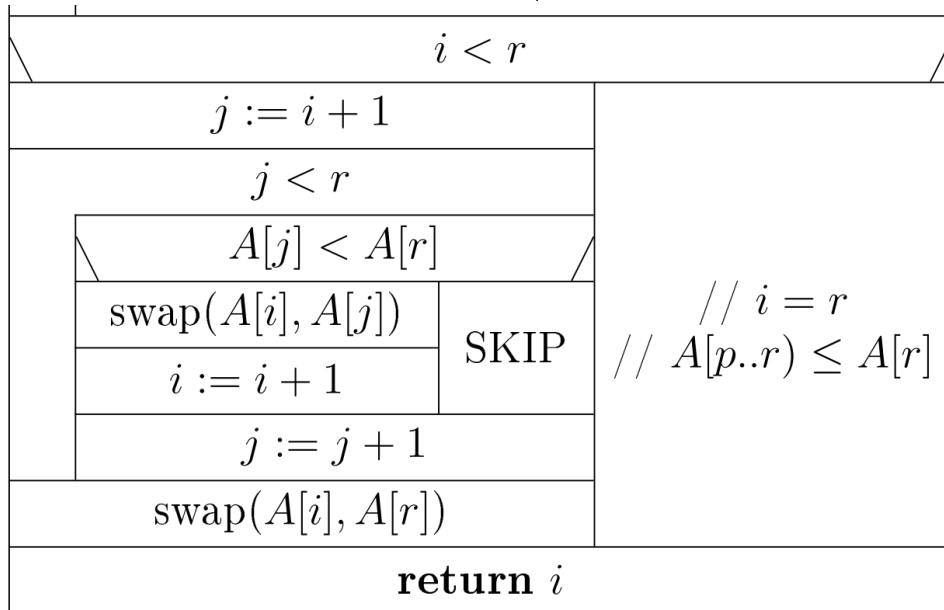
Quicksort - Looking for smaller elements and swap

Set $j := i + 1$:	$\langle 12, \textcolor{blue}{31}, \textcolor{red}{23}, 20, 11, 55, \textcolor{brown}{14} \rangle$
Look for $A[j] < \textcolor{brown}{PIVOT}$:	$\langle 12, \textcolor{blue}{31}, 23, 20, \textcolor{red}{11}, 55, \textcolor{brown}{14} \rangle$
SWAP $A[i]$ and $A[j]$:	$\langle 12, \textcolor{blue}{11}, 23, 20, \textcolor{blue}{31}, 55, \textcolor{brown}{14} \rangle$
Set $i := i + 1$:	$\langle 12, 11, \textcolor{blue}{23}, 20, \textcolor{red}{31}, 55, \textcolor{brown}{14} \rangle$
Continue $A[j] < \textcolor{brown}{PIVOT}$:	$\langle 12, 11, \textcolor{blue}{23}, 20, \textcolor{red}{31}, 55, \textcolor{brown}{14} \rangle$
$i < r$	
$j := i + 1$	
$j < r$	
$A[j] < A[r]$	
swap($A[i], A[j]$)	SKIP $// i = r$ $// A[p..r] \leq A[r]$
$i := i + 1$	
$j := j + 1$	

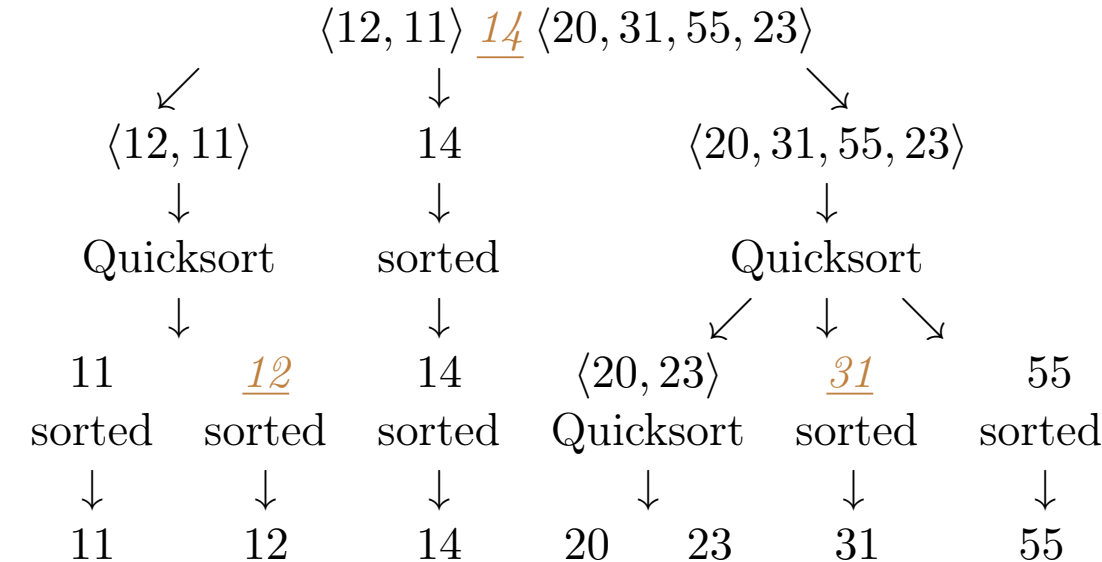
Quicksort - Final

<i>j</i> reaches the <i>PIVOT</i> :	$\langle 12, 11, \underline{23}, 20, 31, 55, \underline{14} \rangle$
SWAP $A[\underline{i}]$ with the <i>PIVOT</i> :	$\langle 12, 11, \underline{14}, 20, 31, 55, \underline{23} \rangle$

$\langle \text{sorted} \rangle$ *PIVOT* $\langle \text{sorted} \rangle$: $\langle 12, 11 \rangle \underline{14} \langle 20, 31, 55, 23 \rangle$

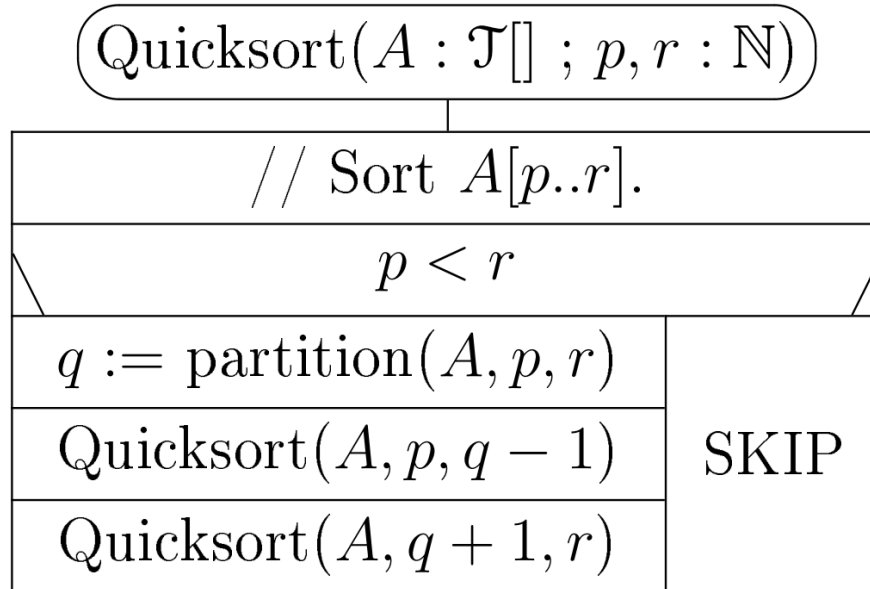


Quicksort - Recursion

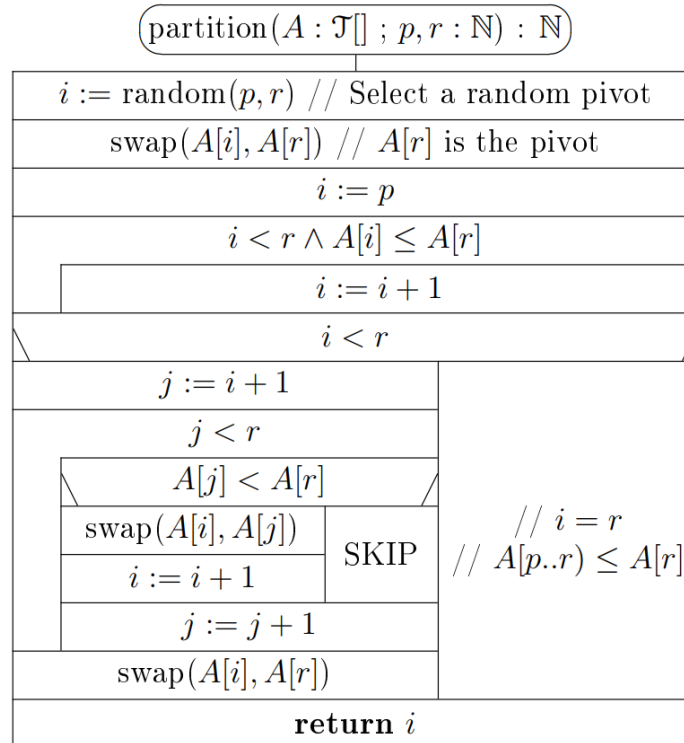


sorted: $\langle 11, 12, 14, 20, 23, 31, 55 \rangle$

Quicksort - Recursion Structogram



Quicksort - Partition Structogram



Quicksort - Summary

Time complexity:

- $mT(n), AT(n) \in O(n * \log(n))$
- $MT(n) \in O(n^2)$: Worst-case scenario with a unbelievably low possibility.

Space complexity

- $O(1)$
- "In-place" algorithm - it sorts the elements within the array with at most a constant amount of variables outside the array at any given time.

Not Stable: Quicksort can be stable but it typically isn't implemented that way. Making it stable either requires order N storage (as in a naive implementation) or a bit of extra logic for an in-place version.

Array example - First partitioning

1	2	3	4	5	6	7	8	9	10
8	12	5	14	10	2	9	17	3	9

Partition(A,1,10)

x=9

1	i	j	4	5	6	7	8	9	10
8	12	5	14	10	2	9	17	3	

Csere(A[i],A[j])

1	2	i	4	5	j	7	8	9	10
8	5	12	14	10	2	9	17	3	

Csere(A[i],A[j])

1	2	3	i	5	6	7	8	j	10
8	5	2	14	10	12	9	17	3	

Csere(A[i],A[j])

1	2	3	4	i	6	7	8	9	j
8	5	2	3	10	12	9	17	14	

tengely a helyére

1	2	3	4	i	6	7	8	9	10
8	5	2	3	9	12	9	17	14	10

Első partícionálás vége

- We pick the last element as the pivot, and leave an 'empty slot' there.
- i starts at the beginning and stops at the element 12, because that's the first larger element. j starts from the element after i, and stops right there because $5 < x$. We swap (A[2],A[3]), and then $i=3$, $j=4$
- j skips every element that is larger or equal, and stops at 2, the first smaller element. Swap(A[3],A[6]), $i=4$, $j=7$
- j until the 9th element, Swap(A[4],A[9]), $i=5$, $j=10$
- j reaches the 'empty slot' and we swap it with the element at i
- Return the value of i

Quicksort - Subsequent runs

1	2	3	4	5	6	7	8	9	10
8	5	2	3	9	12	9	17	14	10

partition(A,1,10)=5

1	2	3	4	5	6	7	8	9	10
8	5	2	3	9	12	9	17	14	10

x=3

Következik: partititon(A,1,4)

tengely:A[4]

1	i	3	4	5	6	7	8	9	10
2	3	8	5	9	12	9	17	14	10

partition(A,1,4)=2

1	i	3	4	5	6	7	8	9	10
2	3	8	5	9	12	9	17	14	10

x=5

Következik: partititon(A,3,4)

tengely:A[4]

1	2	i	4	5	6	7	8	9	10
2	3	5	8	9	12	9	17	14	10

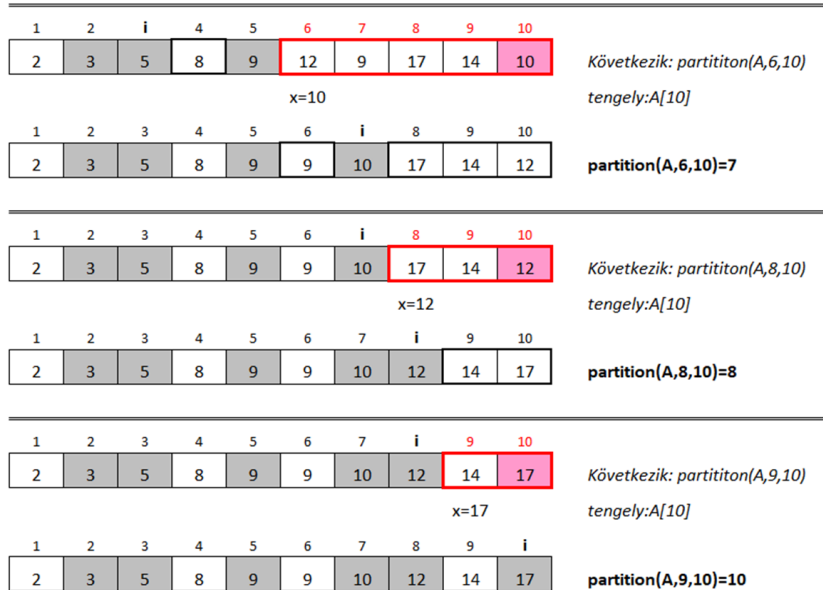
partition(A,3,4)=3

- partition(A,1,10)
8,5,2,3,+9,12,9,17,14,10
Comparisons:
9

- partition(A,1,4)
2,+3,8,5
Comparisons:
3 (Sum=12)

- partition(A,3,4)
+5,8
Comparisons:
1 (Sum=13)

Quicksort - Subsequent runs



- partition(A,6,10):
9,+10,17,14,12
comparisons:
4 (Sum=17)
- partition(A,8,10):
+12,14,17
comparisons:
2 (Sum=19)
- Partition(9,10)
14,+17
comparisons:
1 (Sum=20)

Thank you for your attention!